

ChallengeDaily

Frontend Audit Report

- > Exhaustive code review across 12 categories
- > 75 issues identified · 2 Critical · 19 High

Author: TeleAgent Audit Engine
Date: 2026-07-08

1. React Anti-Patterns & Code Quality

Severity	File:Line	Issue	Suggested Fix
High	Todos.tsx:56, Habits.tsx:49, Countdowns.tsx: 46, Timeline.tsx, Agent.tsx	window.confirm()) for destructive actions - inconsistent with app's custom Toast/UI system, blocks UI thread, no dark-mode styling	Replace with a custom ConfirmDialog component using the existing Toast infrastructure
High	Overview.tsx, AppRecords.tsx	Manual useEffect + useState for data fetching instead of the app's useAsyncData hook - no stale -while-error, no background refresh, no retry logic	Refactor to use useAsyncData for consistency with the rest of the app
Medium	Achievements.ts x:53	setTimeout(hand leCheck, 2000) in useEffect - handleCheck is a closure that captures stale toast reference; no deps array properly tracked	Move handleCheck into the effect or use useCallback with proper deps

Medium	WeekPlan.tsx:197	<pre>window.location .hash = '#/focus?todo_id=' + id - programmatic hash navigation instead of using react-router's navigation</pre>	Use useNavigate() from react-router-dom for SPA navigation
Medium	WeekPlan.tsx:307	<pre>onClick={() => { const t = tasks.find(t => t.status !== 'completed'); if (t) startFocus(t.id) }} - shadowed variable 't' in nested arrow function</pre>	Rename inner variable to avoid shadowing: const pending = tasks.find(...)
Medium	Diary.tsx:90	<pre>if (d <= new Date()) - comparing Date objects with <= which converts to string coercion, unreliable across browsers</pre>	Compare timestamps: if (d.getTime() <= Date.now())
Low	Diary.tsx, Countdowns.tsx, Todos.tsx	Multiple useState calls for individual form fields instead of a single object state - causes unnecessary re-renders on each field change	Consider useReducer or a form library for complex forms
Low	Onboarding.tsx:79	<pre>settings: Record - untyped settings object passed to updateSettings</pre>	Define a typed interface for onboarding settings

2. Error Handling

Severity	File:Line	Issue	Suggested Fix
Critical	HistoryReports.tsx:57	<pre>catch (err: any) with console.error - error silently swallowed, user gets no feedback when report generation fails</pre>	Show toast.error() or ApiErrorDisplay with retry option
High	Onboarding.tsx:92	<pre>catch (err) { console.error('Failed to save onboarding settings') } - settings save failure silently ignored, user proceeds without knowing settings weren't saved</pre>	Show toast.error and prevent navigation or offer retry
High	Overview.tsx (manual useEffect pattern)	<pre>useEffect data fetching with catch(e) only shows toast.error - no retry mechanism, no error state for conditional rendering</pre>	Migrate to useAsyncData which has built-in retry and error display

Medium	WeekPlan.tsx:106,118,164,177,192	Multiple catch blocks with only error() toast - no distinction between network errors, auth errors, and validation errors	Use the API client's error classification to show specific messages
Medium	client.ts (throughout)	API errors are caught by many pages but error details are lost - generic 'failed, please retry' messages don't help debugging	Surface server error messages when available (API returns structured error details)
Low	Pet.tsx, HeroInfo.tsx	Several async operations in components without try/catch - unhandled promise rejections possible	Wrap all async operations in try/catch with appropriate fallback

3. Performance

Severity	File:Line	Issue	Suggested Fix
High	Overview.tsx	Multiple sequential API calls in useEffect instead of Promise.all - each awaits before starting the next, adding latency	Use Promise.all to parallelize independent API calls

High	HistoryReports.tsx:19-31	14 parallel API requests on mount (Promise.allSettled for 14 dates) - could overwhelm the backend and cause rate limiting	Implement a batch API endpoint or paginate with lazy loading
Medium	WeekPlan.tsx:64-77	getStreak() called for every habit on every render - $O(n*m)$ comparison with date manipulation inside render	Memoize with useMemo or compute on the server side
Medium	Habits.tsx:60-61	todayLogs filter and Map construction on every render without memoization	Wrap in useMemo with appropriate deps
Medium	DeepInsight.tsx:450 lines	Large component with all sub-components inline - all re-render when any state changes	Extract sub-components and use React.memo for static sections
Medium	Agent.tsx:418 lines, Settings.tsx:1001 lines	Very large components (400-1000 lines) - any state change re-renders the entire component tree	Split into smaller components with memo boundaries

Low	WeekPlan.tsx:100-109	loadWeek makes 3 parallel API calls each time - no caching or deduplication when navigating back and forth between weeks	Implement request caching or use SWR-like stale-while-revalidate pattern
Low	Timeline.tsx:621 lines	Large list rendering without virtualization - if activities grow to hundreds, DOM will be slow	Consider react-window or react-virtualized for large lists

4. Security

Severity	File:Line	Issue	Suggested Fix
Critical	client.ts	JWT token stored in localStorage - accessible to any JS including potential XSS; no token encryption at rest	Store token in httpOnly cookie or use Electron's safeStorage API for encryption at rest
High	Report.tsx, HistoryReports.tsx:144-153	urlTransform filters javascript:/data:/vbscript:/ but allows other dangerous schemes like blob: and file:	Use allowlist approach: only allow http: and https: URLs

Medium	Onboarding.tsx:95	localStorage.setItem('cd_onboarding_done', '1') - onboarding completion stored in localStorage can be trivially bypassed	Store onboarding state on the backend and verify server-side
Medium	HistoryReports.tsx:38	navigator.clipboard.writeText() - clipboard access without user awareness of what's being copied (could include sensitive report data)	Show preview of content being copied or confirm action
Low	client.ts:as any	Several 'as any' type assertions bypass TypeScript's type safety, potentially allowing malformed data to reach the UI	Replace with proper type guards and validation
Low	main.cjs	IPC sender validation exists but only for some channels - not all IPC handlers check message origin	Apply isFromMainWindow check to all remaining IPC handlers

5. Accessibility (a11y)

Severity	File:Line	Issue	Suggested Fix
----------	-----------	-------	---------------

High	All pages	No aria-labels on icon-only buttons (delete, edit, play, etc.) - screen readers announce empty or meaningless button labels	Add aria-label to all icon-only buttons describing their action
High	Onboarding.tsx: 262-274	Custom toggle switch lacks keyboard interaction (Space/Enter), proper aria-pressed, and focus indicator	Use native checkbox with custom styling or add keyboard handlers + focus-visible outline
Medium	Diary.tsx	No aria-label on textarea and input fields - placeholder text is not a substitute for accessible labels	Add aria-label or associated elements
Medium	WeekPlan.tsx	Drag and drop is the primary interaction - no keyboard-accessible alternative for task assignment	Provide keyboard shortcuts or a move-to dropdown for task reassignment
Medium	Heatmap.tsx, Health.tsx	SVG charts lack aria-label and description - screen readers cannot interpret visual data	Add aria-label and aria-description to SVG elements with text summary of data
Medium	Sidebar.tsx	Navigation links have no aria-current='page' to indicate active route to screen readers	Add aria-current='page' to the active nav item

Low	All pages	Focus management after navigation or modal close not implemented - keyboard users may lose their position	Implement focus trapping in modals and focus return after close
Low	Achievements.tsx:91	Emoji used as status indicator (locked vs unlocked) - not conveyed to screen readers	Add aria-label with text description like 'Locked achievement' vs 'Unlocked: [name]'

6. State Management

Severity	File:Line	Issue	Suggested Fix
High	Overview.tsx, AppRecords.tsx vs other pages	Inconsistent data fetching patterns: some pages use useAsyncData, others use manual useEffect+useState - two completely different loading/error/refresh paradigms	Standardize on useAsyncData across all pages
High	Settings.tsx:1001 lines	Massive component with 20+ useState calls - prop drilling and re-render cascade; any settings change re-renders entire page	Split into sub-components with local state; consider useReducer for related state groups

Medium	WeekPlan.tsx:83-98	16 pieces of state in one component - complex interactions between viewMode, weekStart, monthKey states that trigger data reloads	Use useReducer with well-defined actions to coordinate state transitions
Medium	Focus.tsx	Timer state managed in component - if component unmounts and remounts, timer state is lost	Lift timer state to a context or persist to backend for reliability
Low	client.ts	No global cache or state store for API responses - each page fetches independently even for shared data (e.g., settings, profile)	Implement a lightweight client-side cache (Map or zustand store) for shared data

7. Type Safety

Severity	File:Line	Issue	Suggested Fix
High	client.ts (throughout)	Multiple 'as any' type assertions, especially around API response handling and settings objects	Define proper response types and remove all 'as any' casts

High	HistoryReports.tsx:57	catch (err: any) – using 'any' type loses all type information about the error	Use 'unknown' type and type-narrow with instanceof checks
Medium	Onboarding.tsx:79	Record for settings – zero type safety on the most critical data submission	Type the settings object properly with required and optional fields
Medium	Settings.tsx	Multiple 'as any' casts on DOM event handlers and setting values	Properly type event handlers: React.ChangeEvent, etc.
Low	WeekPlan.tsx:506	Type assertion 'as Timer Goal Habit' on select onChange – could be bypassed at runtime	Use a type guard function to validate the value
Low	vite.config.js:43	Vite config uses .js not .ts – no type checking on build configuration	Rename to vite.config.ts for type-safe configuration

8. CSS & Styling

Severity	File:Line	Issue	Suggested Fix
----------	-----------	-------	---------------

High	WeekPlan.tsx:52-56,239,276	Hardcoded hex colors (#2D2D2D, #2a2218, #1a1410, #1E1E1E, etc.) directly in style props instead of CSS variables - breaks dark/light mode theming	Replace with var(--cd-*) CSS variables used in the rest of the app
High	WeekPlan.tsx:59,62	Inline font-size with style={{fontSize: 9}}, style={{fontSize: 7}}, style={{fontSize: 8}} - pixel-based sizes don't scale with user preferences	Use Tailwind text-* classes which respect the font-size scale
Medium	Countdowns.tsx:6, Habits.tsx:6	COLORS array with hardcoded hex values - not theme-aware, may be invisible in certain themes	Use CSS variables or Tailwind color classes
Medium	Todos.tsx:9	PRIORITY_COLORS as Tailwind class strings ('text-red-400', etc.) - array index lookup could throw if priority is out of range	Add bounds checking or use a Map with fallback color

Low	WeekPlan.tsx:55	Inline styles mixed with Tailwind classes on same elements - inconsistent styling approach within single component	Prefer CSS variables via Tailwind for consistency with the rest of the app
Low	App.tsx	animate-fade-in used in several pages but animation definition not verified in Tailwind config - could break if config changes	Ensure animation is defined in tailwind.config.cjs extend section

9. Electron Security

Severity	File:Line	Issue	Suggested Fix
Medium	main.cjs	nodeIntegration : false and contextIsolation: true are set (good), but sandbox: true is also set - some IPC patterns may not work correctly	Verify all IPC handlers work correctly with sandbox mode; test on dev and prod builds

Medium	preload.cjs	contextBridge.exposeInMainWorld correctly used (good), but exposed API surface is broad – all IPC channels are exposed through generic invoke	Consider exposing specific typed methods instead of a generic invoke proxy
Low	main.cjs	Auto-update connects to GitHub releases – no signature verification of update packages before applying	Implement code signing and signature verification for auto-updates
Low	main.cjs	Global shortcut registration (F12 for DevTools) – no user consent mechanism for keyboard shortcut registration	Make global shortcuts configurable and opt-in
Info	main.cjs	IPC sender validation (isFromMainWindow) implemented for sensitive channels – good security practice	No action needed – positive finding
Info	preload.cjs	No direct ipcRenderer exposure – uses contextBridge properly	No action needed – positive finding

10. Build & Deployment

Severity	File:Line	Issue	Suggested Fix
Medium	package.json	No build-time type checking - Vite does not run tsc, so TypeScript errors are only caught in IDE	Add tsc --noEmit to the build script or CI pipeline
Medium	vite.config.js: 39-42	Dev server on fixed port 5173 - collision if another instance is running	Add fallback port logic or use strictPort: false
Medium	scripts/bump-version.cjs	Version bump script only increments patch version - no semver validation for major/minor bumps	Add CLI args for --major, --minor, --patch and validate semver format
Low	vite.config.js	No source map configuration for production builds - debugs poorly in production	Add sourcemap: true for production or configure Sentry source map upload
Low	vite.config.js: 21-29	Manual chunk splitting is good but may become stale as dependencies evolve	Periodically review bundle analyzer output to keep chunks optimal
Info	vite.config.js: 10	base: './' correctly set for Electron relative paths - proper configuration	No action needed

11. User Experience

Severity	File:Line	Issue	Suggested Fix
High	HistoryReports.tsx:52-54	handleGenerate uses 'standard' template and today's date regardless of which report date the user expects – confusing UX	Accept date and template as parameters, or at least show which date will be generated
High	WeekPlan.tsx	No confirmation or undo for drag-and-drop task reassignment – accidental drops are easy and unrecoverable	Add toast with undo option after drag-drop reassignment
Medium	Diary.tsx:67-79	Auto-save not implemented – user must manually click Save; unsaved content lost on navigation	Implement debounced auto-save with visual indicator
Medium	Focus.tsx	No notification when timer completes – user may miss the end of a focus session if window is not visible	Use Electron notification API (already available via IPC) to alert on timer completion
Medium	Sidebar.tsx	No breadcrumb or back navigation on sub-pages – users can get lost in deep navigation	Add breadcrumb or back button on detail pages

Low	Diary.tsx:170	History dates limited to 30 shown (diaryDates.slice(0, 30)) - no way to see older diary entries in the calendar navigation	Add 'Load more' or infinite scroll for diary dates
Low	Habits.tsx	No visual distinction between 'not yet done today' and 'partially done' for multi-target habits	Add progress bar or fraction indicator for partial completion

12. API Client & Network

Severity	File:Line	Issue	Suggested Fix
High	client.ts	No request deduplication - same API call can be made multiple times simultaneously (e.g., navigating back and forth)	Implement request deduplication using a Map of in-flight promises
High	client.ts	Token expiry not handled gracefully - if token expires mid-session, all subsequent requests fail with 401	Implement transparent token refresh on 401 response before retrying the original request

Medium	HistoryReports. tsx:22-31	Promise.allSettled fires 14 parallel requests - no throttle or concurrency limit	Use a concurrency limiter (e.g., p-limit) for batch requests
Medium	client.ts	Retry logic with exponential backoff exists (good) but no jitter - all retrying clients hit the server simultaneously	Add random jitter to retry delay to avoid thundering herd
Low	client.ts	AbortController signal not threaded through all API methods - some long operations can't be cancelled on unmount	Ensure all API methods accept an optional signal parameter
Low	shared.tsx (useAsyncData)	Visibility-based polling runs even when page is in background - wasteful	Use document.visibilityState to pause polling when page is hidden

Summary Statistics

Category	Critical	High	Medium	Low	Info	Total
1. React Anti-Patterns	0	2	3	2	0	7

2. Error Handling	1	2	2	1	0	6
3. Performance	0	2	3	2	0	7
4. Security	1	1	2	2	0	6
5. Accessibility	0	2	3	2	0	7
6. State Management	0	2	2	1	0	5
7. Type Safety	0	2	2	2	0	6
8. CSS & Styling	0	2	2	2	0	6
9. Electron Security	0	0	2	2	2	6
10. Build & Deploy	0	0	3	2	1	6
11. User Experience	0	2	3	2	0	7
12. API Client	0	2	2	2	0	6
TOTAL	2	19	29	22	3	75

Priority Recommendations

Fix Immediately (Critical + High)

1. Replace `window.confirm()` with a custom dialog component (affects 5+ pages)
2. Add user-facing error feedback in `HistoryReports` and `Onboarding` (currently silent failures)
3. Migrate `Overview/AppRecords` from manual `useEffect` to `useAsyncData` for consistency
4. Replace hardcoded hex colors in `WeekPlan` with CSS variables (breaks theming)
5. Add `aria-labels` to all icon-only buttons (accessibility compliance)
6. Implement request deduplication and token refresh in the API client
7. Move JWT token from `localStorage` to `safeStorage` for encryption at rest
8. Use `allowlist` (http/https only) instead of `denylist` for URL filtering in `ReactMarkdown`

Address Soon (Medium)

1. Add keyboard-accessible alternatives for drag-and-drop interactions
2. Implement auto-save with debounce in `Diary` page
3. Add undo/revert for drag-and-drop task reassignment
4. Memoize computed values in `Habits` (streaks, `todayLogs`) and `WeekPlan`
5. Split `Settings.tsx` (1001 lines) and `Agent.tsx` (418 lines) into smaller components
6. Add `tsc --noEmit` to CI/build pipeline for TypeScript verification
7. Reduce parallel API requests in `HistoryReports` (batch or pagination)
8. Standardize error handling to surface server error details to users